

The operation `z = torch.matmul(x, y)` in PyTorch performs **matrix multiplication** (also known as the dot product for 1D tensors) between two tensors `x` and `y`. The behavior of `torch.matmul` depends on the dimensions of the input tensors. Let's break it down:

Key Points:

Matrix Multiplication Rules:

For two matrices x (of shape $m \times n$) and y (of shape $n \times p$), the result z will have shape $m \times p$.

The number of columns in x must match the number of rows in y .

Broadcasting:

If x and y are higher-dimensional tensors (e.g., batches of matrices), `torch.matmul` supports broadcasting.

1D Tensors (Vectors):

If x and y are 1D tensors (vectors), `torch.matmul` computes their dot product, resulting in a scalar.

2D Tensors (Matrices):

If x and y are 2D tensors (matrices), `torch.matmul` performs standard matrix multiplication.

Higher-Dimensional Tensors:

If x and y are higher-dimensional tensors, `torch.matmul` treats them as batches of matrices and performs batched matrix multiplication.

Examples:

1. Matrix Multiplication (2D Tensors):

```
python
import torch
```

Copy

```
# Define two 2D tensors (matrices)
```

```
x = torch.tensor([[1, 2], [3, 4]]) # Shape: 2x2
```

```
y = torch.tensor([[5, 6], [7, 8]]) # Shape: 2x2
```

```
# Perform matrix multiplication
```

```
z = torch.matmul(x, y)
```

```
print(z)
```

Output:

Copy

```
tensor([[19, 22],
        [43, 50]])
```

Explanation:

x is a 2×2 matrix, and y is a 2×2 matrix.

The result z is computed as:

$$z = \begin{bmatrix} 1 \cdot 5 + 2 \cdot 7 & 1 \cdot 6 + 2 \cdot 8 \\ 3 \cdot 5 + 4 \cdot 7 & 3 \cdot 6 + 4 \cdot 8 \end{bmatrix} = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$$

2. Dot Product (1D Tensors):

```
python
```

Copy

```
# Define two 1D tensors (vectors)
x = torch.tensor([1, 2, 3]) # Shape: (3,)
y = torch.tensor([4, 5, 6]) # Shape: (3,)
```

```
# Perform dot product
z = torch.matmul(x, y)
```

```
print(z)
```

Output:

```
tensor(32)
```

Explanation:

x and y are 1D tensors (vectors).

The result z is computed as:

$$z = 1 \cdot 4 + 2 \cdot 5 + 3 \cdot 6 = 4 + 10 + 18 = 32$$

3. Batched Matrix Multiplication (3D Tensors):

```
python
```

```
# Define two 3D tensors (batches of matrices)
```

```
x = torch.tensor([[[[1, 2], [3, 4]], [[5, 6], [7, 8]]]]) # Shape: 2x2x2
```

```
y = torch.tensor([[[[9, 10], [11, 12]], [[13, 14], [15, 16]]]]) # Shape: 2x2x2
```

```
# Perform batched matrix multiplication
```

```
z = torch.matmul(x, y)
```

```
print(z)
```

Output:

```
tensor([[[ 31, 34],
          [ 73, 80]],
```

```
        [[155, 166],
          [211, 226]]])
```

Explanation:

x and y are 3D tensors, where each "slice" along the first dimension is a 2×2 matrix.

The result z is computed as:

$$z[0] = \begin{bmatrix} 1 \cdot 9 + 2 \cdot 11 & 1 \cdot 10 + 2 \cdot 12 \\ 3 \cdot 9 + 4 \cdot 11 & 3 \cdot 10 + 4 \cdot 12 \end{bmatrix} = \begin{bmatrix} 31 & 34 \\ 73 & 80 \end{bmatrix}$$
$$z[1] = \begin{bmatrix} 5 \cdot 13 + 6 \cdot 15 & 5 \cdot 14 + 6 \cdot 16 \\ 7 \cdot 13 + 8 \cdot 15 & 7 \cdot 14 + 8 \cdot 16 \end{bmatrix} = \begin{bmatrix} 155 & 166 \\ 211 & 226 \end{bmatrix}$$

Summary:

`torch.matmul(x, y)` performs matrix multiplication or dot product depending on the dimensions of x and y .

It supports 1D, 2D, and higher-dimensional tensors.

For higher-dimensional tensors, it performs batched matrix multiplication.

Copy

Copy

Copy